JDBC: Complete Study & Practice Notes

Theory + lecture progression + original code + improved code + revision practice

How to study: Read the concept first, trace the code, predict its output, then explain why each JDBC class or method is used. Finally, attempt the practice checklist without looking at the answers.

Important: Code from the lecture reference is retained in sequence where available. Improved examples are clearly labelled. Replace local usernames/passwords with your own setup; never hard-code real credentials in a production application.

Contents

• Part 1 — Theory and lecture concepts

• Part 2 — Code progression from the lecture file

• Part 3 — Safer / improved reference patterns

• Part 4 — Practice questions and final revision checklist

PART 1 — THEORY AND LECTURE CONCEPTS

Lecture flow ■ code explained ■ original and updated versions ■ practice

Based on the uploaded JDBC transcript and JDBC.docx code reference. Successive code stages are explained in lecture order. Added clarifications and safer modern code are identified separately.

How to study with these notes

Move through the pages in order. First read the explanation, then trace the code line by line, then type and run it in Eclipse. The separate code file preserves the lecture-style stages and provides updated versions. The same class name appears in several stages because the lecturer progressively modifies the program; these are not meant to be pasted together.

Contents

1. Why databases when files and serialization exist?

2. The database-driver coupling problem

3. ODBC and JDBC

4. JDBC architecture and specifications

5. Four types of JDBC drivers

6. MySQL Server, Workbench and Connector/J

7. Schema, users, permissions and table setup

8. First JDBC connection: original code

9. Resource cleanup and try-with-resources

10. Statement and ResultSet

11. INSERT, UPDATE and DELETE

12. Transactions and ACID

13. Login example and SQL injection

14. PreparedStatement and placeholders

15. CRUD with reusable PreparedStatements

16. DatabaseMetaData

17. Stored procedures and CallableStatement

18. SQL functions and final practice

1. Why databases when files and serialization exist?

Sir begins by broadening the definition of database work: store data, retrieve it, update it, manipulate it and organise it. A database

is an organised collection of data that can be accessed and managed.

The class had previously stored Employee objects—Manager, Engineer, Salesperson—using serialization. Serialization persists

object state to a file so it can be read back later. The lecturer asks: if files can persist data, why use MySQL, Oracle or MongoDB?

The important difference is not that a database uses no files. A database server ultimately persists data on storage too. It adds

mature capabilities: queries, indexing, constraints, multi-user concurrency, transaction handling, permissions, consistency and a

standard access language such as SQL.

Lecture memory line: the goal is usually to build software that uses a database, not spend a year building a database server from

scratch. A mature server lets developers focus on the application.

Extra clarification: serialization is useful for saving object graphs, but does not automatically provide SQL querying, relational

constraints, concurrent access control or transaction guarantees.

2. The database-driver coupling problem

Imagine an application calling a vendor-specific driver directly. One vendor might expose methods conceptually called

open/query/close; another might use connect/execute/disconnect. If application code depends on those exact methods, changing

the database can force changes throughout the application.

The driver is the adapter that knows how to communicate with its database. The application needs a stable interface so it does not

have to be rewritten every time a vendor or database changes.

Keep the roles distinct: the application contains business logic; the JDBC API provides standard interfaces; the vendor driver

implements those interfaces; the database server executes SQL and manages persistent data.

The method names open/query/close in the lecture are illustrative examples, not literal JDBC methods.

3. ODBC and JDBC

ODBC means Open Database Connectivity. It provides a common interface between applications and database-specific drivers.

The application uses the standard ODBC API while the driver handles vendor-specific communication.

The lecture notes two general limitations of a generic intermediate layer: it may not expose every special feature of every database,

and extra layers may add overhead. Actual support and performance depend on the implementation and workload.

JDBC—Java Database Connectivity—is Java's database-access API. It gives Java programs standard interfaces for working with

databases through drivers that implement the JDBC contracts.

JDBC and ODBC are related in purpose but are separate APIs; JDBC is the Java-oriented interface.

4. JDBC architecture and specifications

Sir emphasises the word specifications. A specification says what behaviour must be available, not necessarily how the

implementation internally achieves it. The lecture analogy is a requirement to write a program that adds two numbers: the

requirement describes the result, while the programmer chooses the implementation.

JDBC specifies how Java applications interact with databases through APIs such as Connection, Statement, PreparedStatement

and ResultSet. A database vendor provides a driver implementing those APIs for its database.

Architecture: Java application → JDBC API → JDBC driver → database server. Connection represents a database session;

Statement objects send SQL; ResultSet represents rows returned by a query.

A Java application can connect to multiple databases if it has the right drivers, URLs, credentials and permissions.

Java application

|

JDBC API

(Connection, Statement, ResultSet...)

|

JDBC driver

|

database server

The JDBC API does not itself store your rows; the database server does.

5. Four types of JDBC drivers

Type 1: JDBC–ODBC bridge. Calls travel through JDBC, ODBC and a native driver. It is a historical design and should not be used

with modern Java; the old bridge was removed from modern JDKs.

Type 2: partly Java, partly native code. It relies on vendor native libraries/client software, so installation and platform configuration

matter. It can access vendor-specific capabilities.

Type 3: pure Java driver that communicates using a network protocol with middleware, which then communicates with the database.

The middle tier can add flexibility but also another hop.

Type 4: pure Java driver that speaks the database's own protocol directly. MySQL Connector/J is commonly a Type 4 driver. It is a

typical choice for modern Java applications.

Sir’s analogy is “chit-chat”: a more indirect/general route may involve more communication, while a direct vendor protocol may be

faster. This is a memory aid, not a guarantee that one driver always wins.

Type

Path

Trade-off

JDBC → ODBC → native driver → DB

Legacy

Java + native client → DB

Native setup

Java → middleware → DB

Extra hop

Pure Java → DB protocol

Common modern choice

Thin-driver and performance descriptions in the lecture are simplified; compare actual driver documentation and workload for real

deployments.

6. MySQL Server, Workbench and Connector/J

The transcript distinguishes MySQL Community Server (the database server), MySQL Shell (a command-line interface), and MySQL Workbench (a GUI client). Workbench sends commands to the server; it is not the JDBC driver.

Java needs Connector/J, the JDBC driver for MySQL. The JAR contains compiled driver classes, including the class named in the lecture: com.mysql.cj.jdbc.Driver.

In Eclipse: right-click project → Properties → Java Build Path → Libraries/Class Path → Add External JARs → select the Connector/J JAR → Apply and Close. Then the Java compiler/runtime can find the driver classes.

Driver versions change. Choose a Connector/J version compatible with your Java version and follow its current installation guidance.

1. Why databases when files and serialization exist?

Sir begins by broadening the definition of database work: store data, retrieve it, update it, manipulate it and organise it. A database is an organised collection of data that can be accessed and managed.

The class had previously stored Employee objects—Manager, Engineer, Salesperson—using serialization. Serialization persists object state to a file so it can be read back later. The lecturer asks: if files can persist data, why use MySQL, Oracle or MongoDB?

The important difference is not that a database uses no files. A database server ultimately persists data on storage too. It adds mature capabilities: queries, indexing, constraints, multi-user concurrency, transaction handling, permissions, consistency and a standard access language such as SQL.

Lecture memory line: the goal is usually to build software that uses a database, not spend a year building a database server from scratch. A mature server lets developers focus on the application.

Extra clarification: serialization is useful for saving object graphs, but does not automatically provide SQL querying, relational constraints, concurrent access control or transaction guarantees.

2. The database-driver coupling problem

Imagine an application calling a vendor-specific driver directly. One vendor might expose methods conceptually called open/query/close; another might use connect/execute/disconnect. If application code depends on those exact methods, changing the database can force changes throughout the application.

The driver is the adapter that knows how to communicate with its database. The application needs a stable interface so it does not have to be rewritten every time a vendor or database changes.

Keep the roles distinct: the application contains business logic; the JDBC API provides standard interfaces; the vendor driver implements those interfaces; the database server executes SQL and manages persistent data.

The method names open/query/close in the lecture are illustrative examples, not literal JDBC methods.

3. ODBC and JDBC

ODBC means Open Database Connectivity. It provides a common interface between applications and database-specific drivers.

The application uses the standard ODBC API while the driver handles vendor-specific communication.

The lecture notes two general limitations of a generic intermediate layer: it may not expose every special feature of every database, and extra layers may add overhead. Actual support and performance depend on the implementation and workload.

JDBC—Java Database Connectivity—is Java's database-access API. It gives Java programs standard interfaces for working with databases through drivers that implement the JDBC contracts.

JDBC and ODBC are related in purpose but are separate APIs; JDBC is the Java-oriented interface.

4. JDBC architecture and specifications

Sir emphasises the word specifications. A specification says what behaviour must be available, not necessarily how the implementation internally achieves it. The lecture analogy is a requirement to write a program that adds two numbers: the requirement describes the result, while the programmer chooses the implementation.

JDBC specifies how Java applications interact with databases through APIs such as Connection, Statement, PreparedStatement and ResultSet. A database vendor provides a driver implementing those APIs for its database.

Architecture: Java application → JDBC API → JDBC driver → database server. Connection represents a database session; Statement objects send SQL; ResultSet represents rows returned by a query.

A Java application can connect to multiple databases if it has the right drivers, URLs, credentials and permissions.

Java application

|

JDBC API

(Connection, Statement, ResultSet...)

|

JDBC driver

|

database server

The JDBC API does not itself store your rows; the database server does.

5. Four types of JDBC drivers

Type 1: JDBC–ODBC bridge. Calls travel through JDBC, ODBC and a native driver. It is a historical design and should not be used

with modern Java; the old bridge was removed from modern JDKs.

Type 2: partly Java, partly native code. It relies on vendor native libraries/client software, so installation and platform configuration

matter. It can access vendor-specific capabilities.

Type 3: pure Java driver that communicates using a network protocol with middleware, which then communicates with the database.

The middle tier can add flexibility but also another hop.

Type 4: pure Java driver that speaks the database's own protocol directly. MySQL Connector/J is commonly a Type 4 driver. It is a

typical choice for modern Java applications.

Sir’s analogy is “chit-chat”: a more indirect/general route may involve more communication, while a direct vendor protocol may be

faster. This is a memory aid, not a guarantee that one driver always wins.

Type

Path

Trade-off

JDBC → ODBC → native driver → DB

Legacy

Java + native client → DB

Native setup

Java → middleware → DB

Extra hop

Pure Java → DB protocol

Common modern choice

Thin-driver and performance descriptions in the lecture are simplified; compare actual driver documentation and workload for real

deployments.

6. MySQL Server, Workbench and Connector/J

The transcript distinguishes MySQL Community Server (the database server), MySQL Shell (a command-line interface), and

MySQL Workbench (a GUI client). Workbench sends commands to the server; it is not the JDBC driver.

Java needs Connector/J, the JDBC driver for MySQL. The JAR contains compiled driver classes, including the class named in the

lecture: com.mysql.cj.jdbc.Driver.

In Eclipse: right-click project → Properties → Java Build Path → Libraries/Class Path → Add External JARs → select the

Connector/J JAR → Apply and Close. Then the Java compiler/runtime can find the driver classes.

Driver versions change. Choose a Connector/J version compatible with your Java version and follow its current installation

guidance.

7. Schema, users, permissions and table setup

The lecture creates a schema named kharghar, creates a database user, grants access, then creates a users table with username,

password, name and email columns. It inserts sample records through Workbench.

Workbench is a GUI that helps create objects, but SQL is still being executed behind the scenes. The GUI can show the SQL that it

will send to the server.

Permissions should be limited to what the application needs. A normal application account should not be given full

database-administrator privileges just because it makes setup easier.

The sample username/password in the supplied code are classroom examples. Use private local credentials and never commit

real passwords to source control.

8. First JDBC connection: original code

The first program declares Connection as null, explicitly loads the driver with Class.forName, calls DriverManager.getConnection,

prints success, catches ClassNotFoundException and SQLException, and closes the connection in finally.

DriverManager.getConnection(url, username, password) tries to establish the connection. The URL in the source is

jdbc:mysql://localhost:3306/kharghar: jdbc:mysql identifies the driver/database scheme, localhost is the host, 3306 is the common

MySQL port, and kharghar is the schema.

Class.forName was common in older JDBC examples. Modern drivers usually register themselves automatically when the JAR is

available, so explicit loading is often optional. The code can still be useful for understanding older examples or unusual setups.

Updated examples below use YOUR_PASSWORD instead of repeating the sample credential.

```
Connection connection = null;
try {
Class.forName("com.mysql.cj.jdbc.Driver");
connection = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/kharghar",
"cdac", "YOUR_PASSWORD");
System.out.println("Success " + connection);
} catch (ClassNotFoundException e) {
```

```
e.printStackTrace();
} catch (SQLException e) {
e.printStackTrace();
} finally {
try {
if (connection != null) connection.close();
} catch (SQLException e) {
e.printStackTrace();
}
}
```

9. Resource cleanup and try-with-resources

JDBC resources consume client and server resources. A connection left open may contribute to connection exhaustion. Statements

and ResultSets should also be closed when no longer needed.

try-with-resources closes objects that implement AutoCloseable when control leaves the block, including when an exception occurs.

Connection, Statement, PreparedStatement and ResultSet support this pattern.

Resources close in reverse order of declaration, so a ResultSet closes before its Statement and the Statement before its

Connection. This is usually simpler and safer than a manual finally block.

AutoCloseable provides close(); try-with-resources handles the cleanup path for you.

10. Statement and ResultSet

Connection.createStatement() creates a Statement used to send SQL. For a SELECT that returns rows, executeQuery returns a

ResultSet.

A ResultSet starts before its first row. Call next() to move to a row; it returns true when a row is available. A while loop continues

until next() returns false.

The source mixes getString(1), getString(4) and named columns such as getString("password"). JDBC column positions start at 1.

Named labels are easier to read and less fragile when column order changes.

Method rule of thumb: executeQuery for row-returning SELECT; executeUpdate for INSERT, UPDATE, DELETE and many DDL

statements; execute when the result type may vary.

Use try-with-resources for ResultSet as well as Statement and Connection.

```
try (Connection con = DriverManager.getConnection(URL, USER, PASSWORD);
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("SELECT userName, name, email FROM users")) {
while (rs.next()) {
System.out.println(rs.getString("userName"));
System.out.println(rs.getString("name"));
System.out.println(rs.getString("email"));
}
} catch (SQLException e) {
e.printStackTrace();
}
```

11. INSERT, UPDATE and DELETE

The lecture reads values from Scanner, concatenates them into an INSERT string and calls executeUpdate. Later it updates a

password for a matching username, and demonstrates deleting a user.

executeUpdate returns the number of affected rows for INSERT/UPDATE/DELETE. Zero may mean the WHERE condition matched

no rows. Always inspect WHERE clauses before running destructive SQL.

String concatenation is useful for seeing how SQL is assembled, but unsafe for untrusted input: apostrophes can break the query

and malicious input can alter its meaning. The later PreparedStatement section is the safer next step.

The code file retains the concatenation examples for study, but labels them as unsafe patterns for real user input.

12. Transactions and ACID

The source contains commented-out setAutoCommit(false) and commit() calls. These introduce transaction control: multiple related

database operations can be treated as one logical unit.

With auto-commit enabled, each statement is usually committed automatically. For a multi-step transaction, disable auto-commit,

execute the related statements, call commit when all succeed, or rollback if an operation fails.

ACID means Atomicity, Consistency, Isolation and Durability. JDBC provides methods to control transactions, while the database

engine and transaction design determine the guarantees.

Use a transactional table engine such as MySQL InnoDB for transaction practice.

13. Login example and SQL injection

The lecture checks whether a row exists for a supplied username and password. If ResultSet.next() is true, the program prints

Authenticated; otherwise it prints Authentication Failure.

The first version concatenates input into SQL. If the user enters SQL syntax, the database may interpret it as part of the command

rather than as plain data. This is SQL injection.

PreparedStatement parameter placeholders help prevent this by separating SQL structure from values. Production authentication

must also avoid plaintext password storage; use a suitable password-hashing algorithm and verify hashes securely.

The concatenation code is included as a historical learning stage only; do not deploy it.

```
String sql = "SELECT userName FROM users WHERE userName=? AND password=?";
try (PreparedStatement ps = con.prepareStatement(sql)) {
ps.setString(1, userName);
ps.setString(2, password);
try (ResultSet rs = ps.executeQuery()) {
System.out.println(rs.next() ? "Authenticated" : "Authentication Failure");
}
}
```

14. PreparedStatement and placeholders

Prepare SQL with question marks, for example SELECT * FROM users WHERE userName=? AND password=?. Then bind values

with setString(1, userName) and setString(2, password).

Parameter numbering begins at 1. The first question mark is parameter 1, the second is parameter 2. The order of setter calls does

not change which placeholder they fill.

Values are sent as parameters, not pasted into the SQL text. This is safer and often makes SQL easier to maintain. Prepared

statements can also be reused for repeated operations.

In the source, password += scanner.nextLine() consumes the remainder of a line after next(). Mixing next() and nextLine() can be

confusing; use one input strategy consistently.

String sql = "UPDATE users SET password=? WHERE userName=?";

try (PreparedStatement ps = con.prepareStatement(sql)) {

ps.setString(1, newPassword); // first ?

ps.setString(2, userName); // second ?

int rows = ps.executeUpdate();

System.out.println("Rows updated: " + rows);

}

15. CRUD with reusable PreparedStatements

The code reference prepares SELECT, INSERT and UPDATE statements. An UPDATE template is UPDATE users SET

password=? WHERE userName=?. Therefore parameter 1 is the new password and parameter 2 is the username.

For INSERT INTO users VALUES(?,?,?,?), bind values in the table's column order. A clearer version names the columns explicitly,

so changes in table layout are less likely to silently break the code.

CRUD stands for Create, Read, Update and Delete. PreparedStatement can support each operation; choose executeQuery for a

row-returning SELECT and executeUpdate for data changes.

Bind each parameter before execution. If an update affects zero rows, check whether the username exists.

16. DatabaseMetaData

connection.getMetaData() returns DatabaseMetaData, which can provide information about the database, tables, columns,

supported features and driver details.

The source calls getTables and getColumns, then prints fields from the returned metadata ResultSets. Prefer documented labels

such as TABLE_NAME, COLUMN_NAME and TYPE_NAME over unexplained numeric positions.

Metadata is useful for database browsers, schema-inspection tools, migration utilities and software that needs to discover table

structures.

Metadata catalog/schema arguments can vary by driver. If no tables appear, check the actual schema name and the driver's

metadata conventions.

DatabaseMetaData meta = con.getMetaData();

try (ResultSet tables = meta.getTables(null, null, "%", new String[]{"TABLE"})) {

while (tables.next()) System.out.println(tables.getString("TABLE_NAME"));

}

17. Stored procedures and CallableStatement

The DOCX defines a procedure named GetRecordCount, but its body selects rows from users, category and products; it does not

actually count records. The procedure name does not determine its behaviour—the SQL body does.

Java calls a procedure through connection.prepareCall("CALL GetRecordCount()"). CallableStatement.execute() indicates whether

the first result is a ResultSet. getMoreResults() advances to later results.

A procedure may return result sets, update counts, output parameters, or a combination. Robust code needs to handle the results

the procedure actually returns.

The supplied procedure is MySQL-specific SQL syntax.

try (CallableStatement call = con.prepareCall("CALL GetRecordCount()")) {

```
boolean hasResult = call.execute();
while (hasResult) {
try (ResultSet rs = call.getResultSet()) {
while (rs != null && rs.next()) System.out.println(rs.getString(2));
}
hasResult = call.getMoreResults();
}
}
```

18. SQL functions and final practice

The final example defines GetPrice(categoryId, productId), selects productPrice + 500 for the matching product and returns the value. Java calls it with SELECT GetPrice(?, ?) and reads the returned value from a ResultSet.

A later version validates categoryId and raises a SQL error using SIGNAL SQLSTATE '45000'. This is MySQL-specific syntax and

shows validation at database level.

Whole-lecture memory flow: database need → driver coupling → JDBC standard → driver types → install Connector/J → Connection → Statement/PreparedStatement → execute SQL → process ResultSet/update count → close resources → transactions, metadata, procedures and functions.

Practice: connect; display rows; insert with PreparedStatement; update; authenticate with placeholders; explain driver types; inspect metadata; call a procedure and function; explain commit/rollback.

```
try (PreparedStatement ps = con.prepareStatement("SELECT GetPrice(?, ?)")) {
ps.setInt(1, 1);
ps.setInt(2, 1);
try (ResultSet rs = ps.executeQuery()) {
if (rs.next()) System.out.println(rs.getFloat(1));
}
}
```

PART 2 — LECTURE CODE PROGRESSION (SOURCE ORDER)

The following code blocks are transcribed from the provided JDBC.docx in the order they appeared. Comments and older approaches are useful for remembering how the program evolved. Some source blocks may be incomplete on their own because the lecturer progressively changed the same program.

Lecture code stage 1

```
package com.pankaj.jdbc;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class Program {
	public static void main(String[] args) {
		// TODO Auto-generated method stub
		Connection connection = null;
		// Code iwll change later
		try {
			Class.forName("com.mysql.cj.jdbc.Driver");
			// MySQL t4
			connection = DriverManager.getConnection("jdbc:mysql://localhost:3306/kharghar","cdac","Cdac@123");
			/*
			 * // Oracle T2 jdbc:oracle:oci:@ServiceID:Port
			 *
			 * // Oracle T4 jdbc:oracle:thin:@IP:1521
			 */
			System.out.println("Success " + connection);
		} catch (ClassNotFoundException e) {
			// TODO Auto-generated catch block
			e.printStackTrace();
		} catch (SQLException e) {
			// TODO Auto-generated catch block
			e.printStackTrace();
		}
		finally
		{
			try {
				if(connection!=null)
					connection.close();
			} catch (SQLException e) {
				// TODO Auto-generated catch block
				e.printStackTrace();
			}
		}
	}
}
```

Lecture code stage 2

```
package com.pankaj.jdbc;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
public class Program {
	public static void main(String[] args) {
		// TODO Auto-generated method stub
				try(Connection connection =
		    DriverManager.getConnection("jdbc:mysql://localhost:3306/kharghar","cdac","Cdac@123");
			Statement stSelect = connection.createStatement();
			ResultSet result = stSelect.executeQuery("select * from users"))
		{
			while(result.next())
			{
				System.out.println(result.getString(1));
				System.out.println(result.getString("password"));
				System.out.println(result.getString("name"));
				System.out.println(result.getString(4));
				System.out.println("***************");
			}
			System.out.println("Success " + connection);
		}  catch (SQLException e) {
			// TODO Auto-generated catch block
			e.printStackTrace();
		}
	}
}
```

Lecture code stage 3

```
package com.pankaj.jdbc;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;
import javax.swing.plaf.synth.SynthOptionPaneUI;
public class Program {
■public static void main(String[] args) {
■■// TODO Auto-generated method stub
                try(Connection connection =
■■    DriverManager.getConnection("jdbc:mysql://localhost:3306/kharghar","cdac","Cdac@123");
■■■Statement stInsert = connection.createStatement();
■■■Scanner scanner = new Scanner(System.in))
■■{
■■■//connection.setAutoCommit(false);
■■■System.out.println("Ente rth username");
■■■String userName = scanner.next();
■■■System.out.println("Enter the passowrd");
■■■String password = scanner.next();
■■■System.out.println("Enter the name");
■■■String name = scanner.next();
■■■System.out.println("Ente r the email");
■■■String email = scanner.next();
■■■String sqlInsert = "insert into users values('" +
■■■userName +"','" + password + "','" + name +"','" + email
■■■■■+"')";
■■■System.out.println(sqlInsert);
■■■stInsert.executeUpdate(sqlInsert);
■■■//connection.commit();
■■■System.out.println("Record Saved");
■■}  catch (SQLException e) {
■■■// TODO Auto-generated catch block
■■■e.printStackTrace();
■■}
■}
}
```

Lecture code stage 4

```
try (Connection connection = DriverManager.getConnection("jdbc:mysql://localhost:3306/kharghar", "cdac",
■■■■"Cdac@123");
■■■■Statement stInsert = connection.createStatement();
■■■■Scanner scanner = new Scanner(System.in)) {
■■■// connection.setAutoCommit(false);
■■■System.out.println("Ente rth username");
■■■String userName = scanner.next();
■■■// stInsert.executeUpdate("delete from users where userName='" + userName +
■■■// "'");
■■■System.out.println("Enter the passowrd");
■■■String password = scanner.next();
■■■stInsert.executeUpdate("update users set password='"
■■■■■+ password +"' where userName='" + userName +"'");
---------------------------------------package com.pankaj.jdbc;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;
import javax.swing.plaf.synth.SynthOptionPaneUI;
public class Program {
■public static void main(String[] args) {
■■// TODO Auto-generated method stub
                try (Connection connection =
■■    DriverManager.getConnection("jdbc:mysql://localhost:3306/kharghar", "cdac",
■■■■"Cdac@123");
■■■■Statement stSelect = connection.createStatement();
■■■■Scanner scanner = new Scanner(System.in)) {
■■■// connection.setAutoCommit(false);
■■■System.out.println("Ente rth username");
■■■String userName = scanner.next();
■■■System.out.println("Enter the passowrd");
■■■String password = scanner.next();
■■■try (ResultSet result = stSelect.executeQuery("select * From users where userName='"
■■■■■+ userName +"' and password='"
■■■■■+ password + "'"))
■■■{
■■■■if(result.next())
■■■■■System.out.println("Authenticated");
■■■■else
■■■■■System.out.println("Authentication Failure");
■■■}
■■■/*
■■■ * System.out.println("Enter the name"); String name = scanner.next();
■■■ *
■■■ * System.out.println("Ente r the email"); String email = scanner.next();
■■■ *
■■■ * String sqlInsert = "insert into users values('" + userName + "','" + password
■■■ * + "','" + name + "','" + email + "')";
■■■ *
■■■ * System.out.println(sqlInsert);
■■■ *
■■■ * stInsert.executeUpdate(sqlInsert);
■■■ */
■■■// connection.commit(); System.out.println("Record Saved");
■■} catch (SQLException e) {
■■■// TODO Auto-generated catch block
■■■e.printStackTrace();
■■}
■}
}
```

Lecture code stage 5

```
package com.pankaj.jdbc;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;
public class Program {
■public static void main(String[] args) {
■■// TODO Auto-generated method stub
■■//PreparedStatement
                try (Connection connection =
■■    DriverManager.getConnection("jdbc:mysql://localhost:3306/kharghar", "cdac",
■■■■"Cdac@123");
                                PreparedStatement psSelect = connection.prepareStatement("select *
■■■■    from users where userName=? and password=?");
■■■■Scanner scanner = new Scanner(System.in)) {
■■■// connection.setAutoCommit(false);
■■■System.out.println("Ente rth username");
■■■String userName = scanner.next();
■■■System.out.println("Enter the passowrd");
■■■String password = scanner.next();
■■■password+=scanner.nextLine();
■■■psSelect.setString(1, userName);
■■■psSelect.setString(2, password);
■■■try (ResultSet result = psSelect.executeQuery())
■■■{
■■■■if(result.next())
■■■■■System.out.println("Authenticated");
■■■■else
■■■■■System.out.println("Authentication Failure");
■■■}
■■■/*
■■■ * System.out.println("Enter the name"); String name = scanner.next();
■■■ *
■■■ * System.out.println("Ente r the email"); String email = scanner.next();
■■■ *
■■■ * String sqlInsert = "insert into users values('" + userName + "','" + password
■■■ * + "','" + name + "','" + email + "')";
■■■ *
■■■ * System.out.println(sqlInsert);
■■■ *
■■■ * stInsert.executeUpdate(sqlInsert);
■■■ */
■■■// connection.commit(); System.out.println("Record Saved");
■■} catch (SQLException e) {
■■■// TODO Auto-generated catch block
■■■e.printStackTrace();
■■}
■}
}
```

Lecture code stage 6

```
package com.pankaj.jdbc;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;
public class Program {
■public static void main(String[] args) {
■■// TODO Auto-generated method stub
■■// PreparedStatement
                try (Connection connection =
■■    DriverManager.getConnection("jdbc:mysql://localhost:3306/kharghar", "cdac",
■■■■"Cdac@123");
■■■■PreparedStatement psSelect = connection
■■■■■■■.prepareStatement("select * from users where userName=? and password=?");
■■■■PreparedStatement psRegisterUser = connection.prepareStatement("insert into users values(?,?,?,?)");
■■■■PreparedStatement psUpdatePassword= connection.prepareStatement(
■■■■■■■"update users set password=? where userName=?");
■■■■Scanner scanner = new Scanner(System.in)) {
■■■// connection.setAutoCommit(false);
■■■System.out.println("Ente rth username");
■■■String userName = scanner.next();
■■■System.out.println("Enter the passowrd");
■■■String password = scanner.next();
■■■password += scanner.nextLine();
■■■/*
■■■ * System.out.println("Enter the name"); String name = scanner.next();
■■■ *
■■■ * System.out.println("Ente r the email"); String email = scanner.next();
■■■ */
■■■psUpdatePassword.setString(2, userName);
■■■psUpdatePassword.setString(1, password);
■■■/*
■■■ * psRegisterUser.setString(3, name); psRegisterUser.setString(4, email);
■■■ */
■■■psUpdatePassword.executeUpdate();
■■■/*
■■■ * try (ResultSet result = psSelect.executeQuery()) {
■■■ *
■■■ * if (result.next()) System.out.println("Authenticated"); else
■■■ * System.out.println("Authentication Failure");
■■■ *
■■■ * }
■■■ */
■■■/*
■■■ * System.out.println("Enter the name"); String name = scanner.next();
■■■ *
■■■ * System.out.println("Ente r the email"); String email = scanner.next();
■■■ *
■■■ * String sqlInsert = "insert into users values('" + userName + "','" + password
■■■ * + "','" + name + "','" + email + "')";
■■■ *
■■■ * System.out.println(sqlInsert);
■■■ *
■■■ * stInsert.executeUpdate(sqlInsert);
■■■ */
■■■// connection.commit(); System.out.println("Record Saved");
■■} catch (SQLException e) {
■■■// TODO Auto-generated catch block
■■■e.printStackTrace();
■■}
■}
}
```

Lecture code stage 7

```
package com.pankaj.jdbc;
import java.sql.Connection;
import java.sql.DatabaseMetaData;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
public class Program {
■public static void main(String[] args) {
■■// TODO Auto-generated method stub
■■// PreparedStatement
                try (Connection connection =
■■    DriverManager.getConnection("jdbc:mysql://localhost:3306/kharghar", "cdac",
■■■■■"Cdac@123");
■■■■■)
■■{
■■■DatabaseMetaData dmd = connection.getMetaData();
■■■ResultSet resultTables = dmd.getTables(null, "kharghar", null, new String[] {"TABLE"});
■■■while(resultTables.next())
■■■{
■■■■System.out.println(resultTables.getString(1));
■■■■System.out.println(resultTables.getString(2));
■■■■System.out.println(resultTables.getString(3));
■■■■System.out.println(resultTables.getString(4));
■■■■System.out.println("-------------------------------------");
■■■}
■■■ResultSet resultColumns = dmd.getColumns(null, null, "users", null);
■■■while(resultColumns.next())
■■■{
■■■■for(int tmp=1;tmp<=6;tmp++)
■■■■■System.out.println(resultColumns.getString(tmp));
■■■■System.out.println("^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^");
■■■}
■■} catch (SQLException e) {
■■■// TODO Auto-generated catch block
■■■e.printStackTrace();
■■}
■}
}
```

Lecture code stage 8

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `GetRecordCount`()
BEGIN
■select * from users;
    select * from category;
    select * from products;
END
package com.pankaj.jdbc;
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
public class Program {
■public static void main(String[] args) {
■■// TODO Auto-generated method stub
■■// PreparedStatement
■■try (Connection connection = DriverManager.getConnection("jdbc:mysql://localhost:3306/dac_feb", "cdac",
■■■■■"Cdac@123");
■■■■CallableStatement callable = connection.prepareCall("call GetRecordCount()");
■■■■■)
■■{
■■■boolean status = callable.execute();
■■■while(status)
■■■{
■■■■ResultSet result = callable.getResultSet();
■■■■while(result.next())
■■■■{
■■■■■System.out.println(result.getString(2));
■■■■}
■■■■System.out.println("-----------------------------------------------");
■■■■status = callable.getMoreResults();
■■■}
■■} catch (SQLException e) {
■■■// TODO Auto-generated catch block
■■■e.printStackTrace();
■■}
■}
}
```

Lecture code stage 9

```
CREATE DEFINER=`root`@`localhost` FUNCTION `GetPrice`(categoryId INTEGER, productId INTEGER) RETURNS
     float
 DETERMINISTIC
BEGIN
 declare finalPrice float;
 select (productPrice+500) into finalPrice from products where
 products.categoryId=categoryId and products.productId=productId;
RETURN finalPrice;
END
package com.pankaj.jdbc;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
public class Program {
■public static void main(String[] args) {
■■// TODO Auto-generated method stub
■■// PreparedStatement
■■try (Connection connection = DriverManager.getConnection("jdbc:mysql://localhost:3306/dac_feb", "cdac",
■■■■"Cdac@123");
■■■PreparedStatement psFunction = connection.prepareStatement("select GetPrice(?,?)");
■■■■)
■■{
■■■psFunction.setInt(1,1);
■■■psFunction.setInt(2, 1);
■■■boolean status = psFunction.execute();
■■■if(status)
■■■{
■■■■ResultSet result = psFunction.getResultSet();
■■■■if(result.next())
■■■■■System.out.println(result.getFloat(1));
■■■}
■■} catch (SQLException e) {
■■■// TODO Auto-generated catch block
■■■e.printStackTrace();
■■}
■}
}
================================================
CREATE DEFINER=`root`@`localhost` FUNCTION `GetPrice`(categoryId INTEGER, productId INTEGER) RETURNS
     float
 DETERMINISTIC
BEGIN
 declare finalPrice float;
 if categoryId < 0 then
 signal sqlstate '45000' set message_text='Invalid category id';
 end if;
 select (productPrice+500) into finalPrice from products where
 products.categoryId=categoryId and products.productId=productId;
RETURN finalPrice;
END
================================================
```

PART 3 — IMPROVED REFERENCE PATTERNS

1. Connect to MySQL with automatic resource cleanup

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class JdbcConnectionDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/kharghar";
        String user = "YOUR_USERNAME";
        String password = "YOUR_PASSWORD";

        try (Connection con = DriverManager.getConnection(url, user, password)) {
            System.out.println("Connected successfully: " + !con.isClosed());
        } catch (SQLException e) {
            System.err.println("Database connection failed: " + e.getMessage());
        }
    }
}
```

2. Read rows using Statement and ResultSet (for a fixed query)

```
String sql = "SELECT id, name FROM users";
try (Connection con = DriverManager.getConnection(url, user, password);
     java.sql.Statement st = con.createStatement();
     java.sql.ResultSet rs = st.executeQuery(sql)) {
    while (rs.next()) {
        System.out.println(rs.getInt("id") + " " + rs.getString("name"));
    }
}
```

3. Insert safely using PreparedStatement

```
String sql = "INSERT INTO users(name, password) VALUES (?, ?)";
try (Connection con = DriverManager.getConnection(url, user, password);
     java.sql.PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, name);
    ps.setString(2, passwordHash); // Store a secure hash, not a plain password.
    int rows = ps.executeUpdate();
    System.out.println("Rows inserted: " + rows);
}
```

4. Update and delete with parameters

```
String updateSql = "UPDATE users SET name = ? WHERE id = ?";
try (Connection con = DriverManager.getConnection(url, user, password);
     java.sql.PreparedStatement ps = con.prepareStatement(updateSql)) {
    ps.setString(1, newName);
    ps.setInt(2, userId);
    System.out.println("Rows updated: " + ps.executeUpdate());
}

String deleteSql = "DELETE FROM users WHERE id = ?";
try (Connection con = DriverManager.getConnection(url, user, password);
     java.sql.PreparedStatement ps = con.prepareStatement(deleteSql)) {
    ps.setInt(1, userId);
    System.out.println("Rows deleted: " + ps.executeUpdate());
}
```

5. Transaction: commit or rollback

```
try (Connection con = DriverManager.getConnection(url, user, password)) {
    con.setAutoCommit(false);
    try (java.sql.PreparedStatement ps =
             con.prepareStatement("UPDATE accounts SET balance = balance - ? WHERE id = ?")) {
        ps.setBigDecimal(1, amount);
        ps.setInt(2, fromAccountId);
        ps.executeUpdate();
        // Execute the matching credit operation here.
        con.commit();
    } catch (SQLException e) {
        con.rollback();
        throw e;
    }
}
```

6. Stored procedure call pattern

```
try (Connection con = DriverManager.getConnection(url, user, password);
     java.sql.CallableStatement cs = con.prepareCall("{call procedure_name(?, ?)}")) {
    cs.setInt(1, inputId);
    cs.registerOutParameter(2, java.sql.Types.VARCHAR);
    cs.execute();
    String result = cs.getString(2);
    System.out.println(result);
}
```

7. Quick method guide

```
executeQuery(sql)   -> SELECT; returns ResultSet
executeUpdate(sql)  -> INSERT / UPDATE / DELETE; returns affected-row count
execute(sql)        -> general execution; returns whether first result is a ResultSet
prepareStatement   -> parameterized SQL; use ? placeholders
prepareCall        -> stored procedure call
next()              -> move ResultSet cursor to next row
commit() / rollback()-> finish or undo a transaction
```

PART 4 — PRACTICE AND REVISION

1. Explain in your own words

• Why is JDBC needed when a database already exists?

• What role does the JDBC driver play between Java and MySQL?

• What is the difference between Connection, Statement, PreparedStatement and ResultSet?

• Why should database resources be closed? What does try-with-resources do?

• Explain executeQuery(), executeUpdate() and execute().

• What is SQL injection? How does PreparedStatement reduce the risk?

• What are auto-commit, commit and rollback?

• What is the purpose of DatabaseMetaData?

• When would you use CallableStatement?

2. Trace the code

• If ResultSet contains 3 rows, how many times does while(rs.next()) execute its body?

• If executeUpdate() returns 0 for an UPDATE statement, what does that tell you?

• In setString(1, name), why does the index start at 1 rather than 0?

• What can happen if a query is built by concatenating user input into SQL?

• If an exception occurs before commit(), what should the transaction code do?

3. Mini coding tasks

• Write a program outline that connects to the kharghar database and prints a success message.

• Select all users and print id and name using ResultSet column labels.

• Insert a row using PreparedStatement and two placeholders.

• Update one user by id and print the number of affected rows.

• Write a transaction outline that rolls back if any SQL operation fails.

4. Answers / self-check points

• JDBC is the Java API for database access; the driver implements communication with a specific database.

• Connection represents a database session; Statement sends SQL; PreparedStatement binds parameters; ResultSet represents query rows.

• executeQuery is for queries returning rows; executeUpdate is for data changes and returns a row count; execute is the general-purpose method.

• PreparedStatement separates SQL structure from values and helps prevent user input from being interpreted as SQL code.

• With auto-commit disabled, explicitly commit successful work or rollback failed work.

• ResultSet cursor begins before the first row; call next() before reading each row.

• Parameter indexes in JDBC start at 1.

• DatabaseMetaData exposes information about the database, driver, tables, and supported features.

• CallableStatement is used for stored procedures and can handle IN/OUT parameters.

Final checklist

■ I can explain the JDBC architecture and the role of the driver.

■ I can connect to MySQL and explain the URL, username and password.

■ I can use try-with-resources for Connection, Statement and ResultSet.

■ I can select rows and navigate ResultSet with next().

■ I can distinguish executeQuery, executeUpdate and execute.

■ I can write INSERT, UPDATE and DELETE using PreparedStatement.

■ I can explain SQL injection and why parameterized SQL is safer.

■ I can explain transaction commit/rollback and ACID at a basic level.

■ I can describe DatabaseMetaData, CallableStatement and stored procedures.